



Mansoura University  
Faculty of Computers and Information



# Operating Systems

## Protection

**Dr. Mohamed Handosa**

Assistant Professor of Computer Science

[handosa@mans.edu.eg](mailto:handosa@mans.edu.eg)

# Goals of Protection

- A computer system is a collection of **objects**.
- Each object is accessed via **well-defined operations**.
- **Protection** ensures **correct access** by authorized processes only.
- Goals include:
  - Enforcing permitted access.
  - Preventing unauthorized access.
  - Maintaining system stability and confidentiality.

# Principles of Protection

- **Least Privilege**

- Give each process/user the minimum rights needed.
- Reduces risk if the process is buggy or compromised.

- Can be:

- **Static:** Assigned at process start.
- **Dynamic:** Adjusted during execution (e.g., privilege escalation).

# Grain of Privilege

- **Coarse-Grained Protection:**
  - Easier to manage, fewer access levels.
  - Example: Unix user/root model.
- **Fine-Grained Protection:**
  - More complex but allows more precise control.
  - Example: File ACLs, RBAC, procedure-level domains.

# Domain of Protection

- A **domain** is a set of access rights.
- Each right is a pair: **<object-name, rights-set>**.
- A system may define domains:
  - Per user
  - Per process
  - Per procedure (fine-grained)

# Domain Switching in Unix

- Domain = user identity (UID).
- Switching domains:
  - **setuid** bit on executables → changes UID temporarily.
  - **su** command: switch to another user (password required).
  - **sudo**: execute one command as another domain if permitted.

# Access Matrix

- The **Access Matrix** is a model for representing protection:
  - **Columns:** objects (files, devices, memory)
  - **Rows:** subjects (users, processes, domains)
  - **Cells:** access rights (e.g., read, write, execute)
- Example:

|        | File A      | File B      | Printer |
|--------|-------------|-------------|---------|
| User 1 | read, write | read        | —       |
| User 2 | —           | read, write | print   |
| Admin  | full access | full access | control |

# Access Matrix Implementation (1 of 2)

- **Global Table**

- Store all (subject, object, rights) tuples in one central list.
- Easy to implement, but inefficient for large systems.

- **Access Lists (ACLs)**

- For each **object**, keep a list of all subjects and their rights.
- Example: **File A** → {**User 1: read/write, User 2: read**}
- Good for checking who can access an object.
- Less efficient for revoking a subject's access across all objects.



# Access Matrix Implementation (2 of 2)

- **Capability Lists**

- For each **subject**, store a list of all objects it can access + the rights.
- Example: **User 1** → {**File A: read/write, Printer: print**}
- Efficient for tracking a user's permissions.
- Revocation is harder (must search all users for rights to an object).

- **Lock-Key Mechanism**

- Each object has a list of **locks**.
- Each domain is given a set of **keys**.
- A domain can access an object if one of its keys matches a lock on the object.
- Hardware-friendly, compact implementation of protection.

# Access Control Lists (ACLs)

- Enforces access based on identity and rights.
- Widely supported in modern file systems (e.g., Windows NTFS).
- ACL entries can support:
  - Individual users
  - Groups
  - Wildcards (e.g., "others")

# Role-Based Access Control (RBAC)

- Simplifies management in large systems.
- Users are assigned roles, and roles are assigned permissions.
- Examples:
  - Role: **Admin** → full system access
  - Role: **Editor** → read/write documents
- Roles can reflect job titles, organizational hierarchy, or system needs.

# Revocation of Access Rights

- Revocation refers to removing access that was previously granted.
- Types of Revocation:
  - Immediate versus Lazy
    - **Immediate:** takes effect instantly.
    - **Lazy:** rights are removed later, e.g., when subject logs out.
  - Selective versus Total
    - **Selective:** remove specific rights.
    - **Total:** remove all rights to an object or from a subject.
  - Revocation by Owner
    - Often the object's owner can revoke access.

# Capability-Based Systems

- A capability is a token that names an object and specifies access rights.
- Characteristics:
  - Can be passed to others (delegation).
  - Stored in capability tables, indexed by subject.
  - Capabilities are unforgeable and securely managed.
- Example Systems:
  - **Hydra**: supports fine-grained, dynamic protection with flexible rights.
  - **CAP**: one of the earliest hardware/software systems using capability-based access.
- Efficient and powerful, but revocation and capability transfer are complex.

# Language-Based Protection

- Enforces protection policies at compile or runtime.
- Helps isolate components and prevent insecure access patterns.
- Example: Java
  - Java programs run in the Java Virtual Machine (JVM).
  - A **SecurityManager** defines what the program can access (network, etc.).
  - Uses sandboxing to prevent untrusted code from affecting the host system.

# Thank You